SYSTEM DESIGN ANSWERS

Real-Time Order & Job Tracking System

Dokumen ini memisahkan kondisi yang benar-benar sudah ada pada kode dari rancangan pengembangan berikutnya. Pembahasan tidak mengklaim Redis, replay otomatis, atau idempotency request sudah tersedia.

1. Strategi Real-Time

Implementasi saat ini. Sistem menggunakan WebSocket (Gorilla WebSocket) pada endpoint `/ws/orders/:id`. Browser membuka koneksi untuk satu order. Ketika status order berhasil diperbarui, backend mengambil event terbaru dan memanggil fungsi broadcast ke seluruh koneksi yang terdaftar untuk order tersebut. Frontend menampilkan indikator koneksi dan memperbarui status serta riwayat saat menerima pesan.

Alasan pemilihan. WebSocket menyediakan koneksi dua arah yang tetap terbuka. Server dapat mengirim perubahan segera setelah terjadi tanpa browser harus melakukan polling berkala. Ini cocok untuk pelacakan progres order yang membutuhkan pembaruan cepat dan mengurangi request polling yang berulang.

Batasan. Daftar koneksi berada di memori satu proses backend. WebSocket pada versi ini cocok untuk demo/satu instance, belum untuk deployment multi-instance. Autentikasi koneksi dan pembatasan origin juga perlu diperkuat.

2. Missed Events dan Reconnection

Implementasi saat ini. Backend menyimpan event order di tabel `order_events` dan menyediakan `GET /api/orders/:id/events` dengan urutan `sequence ASC`. Frontend mengambil detail order dan riwayat saat order dimuat. Namun, frontend belum mempunyai mekanisme reconnect otomatis, heartbeat, maupun replay event berdasarkan sequence terakhir setelah koneksi terputus.

Rancangan yang disarankan. Klien menyimpan sequence terakhir yang berhasil diterapkan. Ketika WebSocket terputus, klien mencoba reconnect dengan exponential backoff (misalnya 1, 2, 4, 8 detik, dibatasi maksimum), disertai jitter. Setelah koneksi pulih, klien meminta event dengan sequence lebih besar dari sequence terakhir melalui endpoint history yang menerima parameter `after_sequence`, lalu melanjutkan menerima pesan live. Jika endpoint belum mendukung filter tersebut, klien dapat mengambil seluruh riwayat dan menyaring sequence yang lebih besar secara lokal.

Jaminan yang dituju. WebSocket menjadi jalur notifikasi cepat, sedangkan database/event history menjadi sumber kebenaran. Dengan demikian, pesan live yang terlewat dapat dipulihkan dari penyimpanan, bukan mengandalkan koneksi yang selalu tersedia.

3. Idempotency dan Event Ordering

Implementasi saat ini. Model `OrderEvent` memiliki field `Sequence`, dan order menyimpan `LastSequence`. Perubahan status mencatat event dengan sequence berikutnya dalam transaksi database. Endpoint riwayat mengurutkan event berdasarkan sequence. Model juga memiliki field `IdempotencyKey` dengan unique index, tetapi handler create/update belum membaca atau menggunakan key tersebut untuk mencegah pemrosesan request duplikat.

Rancangan idempotency. Klien mengirim key unik untuk setiap operasi logis (misalnya header `Idempotency-Key`). Backend menyimpan key, identitas operasi, dan hasil respons dalam transaksi yang sama dengan perubahan order. Jika key yang sama dikirim kembali, backend mengembalikan hasil operasi pertama tanpa membuat event atau mengubah status lagi. Key yang sama dengan payload berbeda harus ditolak.

Ordering. Sequence meningkat per order. Klien sebaiknya hanya menerapkan event dengan sequence lebih besar dari sequence terakhir; sequence yang sama/lebih kecil diabaikan. Jika menerima lompatan (contoh terakhir 5 lalu menerima 7), klien mengambil event 6 dan seterusnya dari REST history sebelum melanjutkan. Untuk ketahanan terhadap request bersamaan, sequence perlu dialokasikan secara atomik di transaksi database, misalnya dengan row lock atau conditional update yang mengembalikan nilai sequence baru.

4. Concurrency Control

Implementasi saat ini. Endpoint update status menerima `expected_version`. Backend membaca order lalu menjalankan update bersyarat `WHERE id = ? AND version = ?`. Jika tidak ada baris yang berubah, request ditolak sebagai konflik. Perubahan status dan pembuatan event dilakukan di dalam transaksi database. Pola ini merupakan optimistic concurrency control: beberapa aktor boleh membaca versi yang sama, tetapi hanya update pertama yang cocok dengan versi tersebut yang berhasil.

Contoh. Dua admin membaca order versi 3. Admin A mengubah status dengan expected_version 3 dan berhasil menaikkan versi menjadi 4. Admin B masih mengirim expected_version 3; update bersyarat tidak menemukan baris yang cocok sehingga backend mengembalikan konflik. Klien B harus memuat ulang order dan memutuskan ulang tindakannya.

Catatan perbaikan. Pengambilan order, pemeriksaan transisi, alokasi sequence, update versi, dan insert event harus tetap berada dalam satu transaksi. Untuk menghindari race pada pembacaan status/sequence, pertimbangkan row-level locking atau satu conditional update yang mengembalikan nilai terbaru.

5. Scaling dan Multi-Instance

Kondisi saat ini. Broadcast menggunakan map koneksi WebSocket di memori proses. Jika ada dua instance backend, koneksi klien tersebar di instance berbeda dan broadcast dari instance A tidak otomatis sampai ke klien pada instance B. Redis Pub/Sub belum diimplementasikan.

Rancangan skala horizontal. Setiap instance backend mempertahankan koneksi WebSocket lokal. Setelah perubahan order tersimpan, instance menerbitkan event ke channel Redis, misalnya `order-events`. Semua instance berlangganan channel tersebut dan meneruskan event hanya ke koneksi lokal yang mengikuti order terkait. Load balancer dapat mendistribusikan koneksi WebSocket; sticky session dapat membantu operasional, tetapi Redis tetap diperlukan untuk fanout lintas instance.

Keandalan. Redis Pub/Sub bersifat pesan sementara dan tidak menggantikan penyimpanan event. Event history PostgreSQL tetap menjadi sumber pemulihan. Untuk kebutuhan delivery yang lebih kuat, gunakan transactional outbox: simpan perubahan order, event, dan record outbox dalam satu transaksi; worker menerbitkan outbox ke broker dan menandainya selesai. Konsumen tetap harus idempotent karena pengiriman dapat berulang.

6. Auditability dan Event History

Implementasi saat ini. Tabel `order_events` menyimpan order ID, sequence, status lama/baru, actor ID, note, dan timestamp. Pembuatan order serta perubahan status utama membuat record event. Endpoint `GET /api/orders/:id/events` menyediakan riwayat terurut. Operasi perubahan order dan insert event dilakukan dalam transaksi agar perubahan status tidak tersimpan tanpa event terkait.

Nilai audit. Riwayat membantu menjawab status sebelumnya, status berikutnya, urutan kejadian, aktor yang dicatat, dan waktu perubahan. Data ini juga dapat dipakai untuk membangun ulang tampilan timeline dan memulihkan event yang terlewat oleh klien.

Kekurangan dan pengembangan. Saat ini actor ID berasal dari request dan belum diverifikasi melalui autentikasi, sehingga belum dapat dianggap identitas audit yang tepercaya. Tambahkan identitas aktor dari token/session terverifikasi, tipe aksi, metadata perubahan, correlation/request ID, dan kebijakan retensi. Batasi hak update/delete pada event agar audit trail tidak mudah dimanipulasi. Untuk event yang dikirim ke broker, gunakan outbox agar perubahan database dan niat publikasi tercatat atomik.

Ringkasan Implementasi vs Rencana

Topik	Sudah ada di kode	Belum ada / rencana
Real-time	WebSocket satu instance	Redis fanout, auth koneksi, origin allowlist
Missed events	Event history REST	Reconnect + replay otomatis / after_sequence
Idempotency	Field IdempotencyKey pada model	Deduplikasi request di handler
Ordering	Sequence per order dan sorting history	Gap detection/replay client, alokasi atomik yang diperkuat
Concurrency	expected_version + conditional update	Pengujian race dan row locking bila diperlukan

Topik	Sudah ada di kode	Belum ada / rencana
Scaling	Satu proses backend	Redis Pub/Sub, outbox, multi-instance
Auditability	OrderEvent + transaksi	Aktor terverifikasi dan kontrol audit produksi

Kesimpulan. Versi proyek yang diperiksa sudah mendemonstrasikan REST API, PostgreSQL, event history, optimistic concurrency dasar, dan WebSocket broadcast pada satu instance. Idempotency request, reconnect/replay otomatis, Redis Pub/Sub, outbox, serta autentikasi belum diimplementasikan dan harus dinyatakan sebagai keterbatasan atau pekerjaan lanjutan.